

Pandas vs Polars

Flights Dataset

[Enlace descarga](#)

Polar Expressions

[Enlace de Descarga](#)

```
pip install polars
```

```
import pandas as pd
import polars as pl
import time
```

Pandas test

```
start = time.time()
df = pd.read_csv('flights.csv', dtype={"TAIL_NUMBER": "string", "ORIGIN_AIRPORT": "string", "DESTINATION_AIRPORT": "string"})
df = df[(df['MONTH'] == 12) & (df['ORIGIN_AIRPORT'] == 'SEA') & (df['DESTINATION_AIRPORT'] == 'DFW')]
end = time.time()
print(end - start)
df
```

6.601531744003296

	YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBER	TAIL_NUMBER	ORIGIN_AIRPORT	DESTINATION_AIRPORT	SCHEDULED_DEI
5339849	2015	12	1	2	AA	1230	N3KSAA	SEA	DFW	
5339922	2015	12	1	2	AA	1228	N3AMAA	SEA	DFW	
5341203	2015	12	1	2	AS	658	N302AS	SEA	DFW	
5341275	2015	12	1	2	AA	1308	N3BDAA	SEA	DFW	
5343046	2015	12	1	2	AA	143	N3FCAA	SEA	DFW	
...	...	...	...	...	...	...	...	...	...	...
5810434	2015	12	31	4	AA	1489	N3HTAA	SEA	DFW	
5811917	2015	12	31	4	AA	1402	N3GEAA	SEA	DFW	
5813075	2015	12	31	4	AS	662	N469AS	SEA	DFW	
5813463	2015	12	31	4	AA	1512	N3DLAA	SEA	DFW	
5819038	2015	12	31	4	AS	660	N407AS	SEA	DFW	

319 rows x 31 columns

Polars test

El método `read_csv()` utiliza el modo de ejecución inmediata lo que significa que cargará directamente todo el conjunto de datos antes de realizar cualquier filtrado. En este aspecto, este bloque de código es similar al que utiliza Pandas pero mucho más eficiente

```
start = time.time()
df = pl.read_csv('flights.csv').filter(
    (pl.col('MONTH') == 12) & (pl.col('ORIGIN_AIRPORT') == 'SEA') & (pl.col('DESTINATION_AIRPORT') == 'DFW'))
end = time.time()
print(end - start)
display(df)
```

0.6479246616363525

YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBER	TAIL_NUMBER	ORIGIN_AIRPORT	DESTINATION_AIRPORT	SCHEDULED_DEPARTURE
i64	i64	i64	i64	str	i64	str	str	str	i64
2015	12	1	2	"AA"	1230	"N3KSAA"	"SEA"	"DFW"	5
2015	12	1	2	"AA"	1228	"N3AMAA"	"SEA"	"DFW"	500
2015	12	1	2	"AS"	658	"N302AS"	"SEA"	"DFW"	640
2015	12	1	2	"AA"	1308	"N3BDAA"	"SEA"	"DFW"	645
2015	12	1	2	"AA"	143	"N3FCAA"	"SEA"	"DFW"	830
...	...	...	...	...	...	...	...	...	...
2015	12	31	4	"AA"	1489	"N3HTAA"	"SEA"	"DFW"	1015
2015	12	31	4	"AA"	1402	"N3GEAA"	"SEA"	"DFW"	1155
2015	12	31	4	"AS"	662	"N469AS"	"SEA"	"DFW"	1310
2015	12	31	4	"AA"	1512	"N3DLAA"	"SEA"	"DFW"	1340
2015	12	31	4	"AS"	660	"N407AS"	"SEA"	"DFW"	2350

En la siguiente versión sustituye el método `read_csv()` por `scan_csv()` que **sí utiliza ejecución diferida**.

`scan_csv()` retrasa la ejecución hasta que se llama al método `collect()`, analiza todas las operaciones intermedias e intenta optimizar la operación

```
start = time.time()
df = pl.scan_csv('flights.csv').filter(
    (pl.col('MONTH') == 11) & (pl.col('ORIGIN_AIRPORT') == 'SEA') & (pl.col('DESTINATION_AIRPORT') ==
    'DFW')).collect()
end = time.time()
print(end - start)
display(df)
```

0.7065532207489014

YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBER	TAIL_NUMBER	ORIGIN_AIRPORT	DESTINATION_AIRPORT	SCHEDULED_DEPARTURE
i64	i64	i64	i64	str	i64	str	str	str	i64
2015	11	1	7	"AS"	658	"N307AS"	"SEA"	"DFW"	640
2015	11	1	7	"AA"	1308	"N3CSAA"	"SEA"	"DFW"	645
2015	11	1	7	"AA"	143	"N3JXAA"	"SEA"	"DFW"	828
2015	11	1	7	"AA"	1489	"N3LRAA"	"SEA"	"DFW"	1008
2015	11	1	7	"AA"	1402	"N3BWAA"	"SEA"	"DFW"	1152
...	...	...	...	...	...	...	...	...	...
2015	11	30	1	"AA"	1489	"N3DCAA"	"SEA"	"DFW"	1015
2015	11	30	1	"AA"	1402	"N3AXAA"	"SEA"	"DFW"	1155
2015	11	30	1	"AS"	662	"N487AS"	"SEA"	"DFW"	1305
2015	11	30	1	"AA"	1512	"N3GDAA"	"SEA"	"DFW"	1340
2015	11	30	1	"AA"	2310	"N3NAAA"	"SEA"	"DFW"	1521

Formato Parquet

La verdadera potencia de *LazyEval* se pone de manifiesto cuanto mayores son los conjuntos de datos y, especialmente, si están en un formato columnar. **Apache Parquet** es un formato de archivo de **almacenamiento en columnas** optimizado para su uso en escenarios de procesamiento masivo de *Big Data*. A diferencia de los formatos de almacenamiento basados en filas, como *CSV*, *Parquet* almacena los datos en columnas, lo que lo hace muy eficiente para operaciones con gran volumen de lectura, especialmente cuando se trata de grandes conjuntos de datos

Este formato en columnas permite una mejor compresión y codificación, ya que los tipos de datos similares se almacenan juntos. Por ejemplo, si se tiene un conjunto de datos con millones de filas y solo se necesita consultar unas pocas columnas, *Parquet* permite leer solo las columnas relevantes (*Predicate Pushdown*), lo que reduce significativamente las operaciones de E/S y mejora el rendimiento. Parquet también admite estructuras de datos anidadas complejas, lo que lo hace ideal para datos semiestructurados como *JSON*

Ejemplo: poda del árbol de búsqueda (*Predicate Pushdown*) en *Polars*

```
lf = pl.scan_parquet("events/*.parquet")
fast = (
    lf.filter(pl.col("country") == "IN")
        .filter(pl.col("ts") >= pl.datetime(2025, 9, 1))
```

```

        .select(["ts", "country", "path"])
        .collect()
    )

```

Ejemplo con datos particionados (Polars Hive Partitioning)

NYC Taxi Dataset

El dataset ocupa 325Mb. Scan multiples ficheros con *lazy evaluation*
[TLC Trip Record Data](#)

```

# Planificar acceso
q1 = (
    pl.scan_parquet("Parquet/**/*.parquet")
    .filter(pl.col("passenger_count") > 0) # Filter
    .with_columns([
        # Derive date/time features
        (pl.col("tpep_dropoff_datetime") - pl.col("tpep_pickup_datetime"))
        .dt.total_minutes()
        .alias("trip_duration_minutes")
    ])
    .group_by("passenger_count")
    .agg([
        pl.mean("trip_duration_minutes"),
        pl.mean("total_amount").alias("avg_fare")
    ])
    .sort("passenger_count")
)

# Ejecutar consulta
r1 = q1.collect()
print(r1)

```

passenger_count	trip_duration_minutes	avg_fare
---	---	---
i64	f64	f64
1	15.547899	27.008371
2	17.243722	30.352485
3	16.934745	29.482887
4	17.764046	32.436805
5	20.555213	26.760919
6	23.465055	26.841099
7	18.3	83.639
8	9.911765	102.342059
9	11.666667	62.724444

Particionado estructurado

Si la estructura de particionado es conocida y acorde con los patrones de búsqueda más frecuentes la poda mejora los tiempos de respuesta drásticamente debido a que la búsqueda se focaliza únicamente en las particiones conteniendo los datos

```

lf = pl.scan_parquet("path/YYYY=2025/MM=10/**/*.parquet")
today = lf.filter(pl.col("DD") == "08").collect()

```

Join de datos en diferentes formatos: *parquet* y *csv*

El siguiente ejemplo incluye un **join** que muestra la capacidad para analizar datasets en diferentes formatos

```

q2 = (
    pl.scan_parquet("Parquet/**/*.parquet")
    .join(pl.scan_csv("taxi_zone_lookup.csv"), left_on="PULocationID", right_on="LocationID")
    .filter(pl.col("total_amount") > 25)
    .group_by("Zone")
    .agg(
        (pl.col("total_amount") /
         (pl.col("tpep_dropoff_datetime") - pl.col("tpep_pickup_datetime")).dt.total_minutes()
        ).mean().alias("cost_per_minute")
    ).sort("cost_per_minute", descending=True)
)

```

```
# Collect the results (executes the query)
r2 = q2.collect()
print(r2)
```

Zone	cost_per_minute
---	---
str	f64
Co-Op City	inf
Whitestone	inf
UN/Turtle Bay South	inf
Eltingville/Annadale/Prince's ...	inf
Pelham Bay Park	inf
...	...
New Dorp/Midland Beach	3.958707
Governor's Island/Ellis Island...	3.238929
Rikers Island	2.802784
Green-Wood Cemetery	1.832354
Crotona Park	1.514066

Conversión de datos en diferentes formatos: *Parquet* y *CSV*

El siguiente ejemplo muestra una transformación *CSV*→*Parquet* con dos **singularidades** esenciales:

1. No requiere de almacenamiento intermedio alguno
2. Tipos de datos explícitos (minimiza problemas típicos de los procesos de migración/integración)

```
lf = pl.scan_csv("raw/*.csv", dtypes={"ts": pl.Datetime, "user_id": pl.Int64})
(
    lf.filter(pl.col("ts") >= pl.datetime(2025, 10, 1))
    .with_columns(pl.col("path").str.slice(0, 64).alias("path_64"))
    .sink_parquet("curated/2025-10.parquet")
)
```